```
import heapq

GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)

class PuzzleNode:
    def __init__(self, state, parent=None, move="", g=0):
        self.state = state
        self.parent = parent
        self.move = move
        self.g = g
        self.h = self.calculate_manhattan_distance()
        self.f = self.g + self.h

    def calculate_manhattan_distance(self):
        distance = 0
        for i in range(9):
            val = self.state[i]
            if val != 0:
                target_x, target_y = (val - 1) // 3, (val - 1) % 3
                current_x, current_y = i // 3, i % 3
                distance += abs(target_x - current_x) + abs(target_y -
current_y)
        return distance

    def __lt__(self, other):
        return self.f < other.f

def get_neighbors(node):
    neighbors = []
    state = list(node.state)
    blank_idx = state.index(0)
    r, c = blank_idx // 3, blank_idx % 3
    moves = [(-1, 0, "Up"), (1, 0, "Down"), (0, -1, "Left"), (0, 1,
"Right")]

    for dr, dc, move_name in moves:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            neighbor_idx = nr * 3 + nc
            new_state = state[:]
            new_state[blank_idx], new_state[neighbor_idx] =
new_state[neighbor_idx], new_state[blank_idx]
            neighbors.append(PuzzleNode(tuple(new_state), node,
move_name, node.g + 1))
    return neighbors

def solve_8_puzzle(start_state):
    open_list = [PuzzleNode(start_state)]
    closed_set = set()

    while open_list:
        current_node = heapq.heappop(open_list)
        if current_node.state == GOAL_STATE:
            path = []
            while current_node.parent:
                path.append((current_node.move, current_node.state))
                current_node = current_node.parent
            return path[::-1]
```

```
            closed_set.add(current_node.state)
            for neighbor in get_neighbors(current_node):
                if neighbor.state not in closed_set:
                    heapq.heappush(open_list, neighbor)
    return None

def print_board(state):
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)

if __name__ == "__main__":
    initial_state = (1, 2, 3, 0, 4, 6, 7, 5, 8)
    print("Initial State:")
    print_board(initial_state)
    solution = solve_8_puzzle(initial_state)
    if solution:
        print(f"Solved in {len(solution)} steps!\n")
        for step, (move, state) in enumerate(solution, 1):
            print(f"Step {step}: Move Blank '{move}'")
            print_board(state)
```

main.py | Visualize | Run ▶

```
import heapq
GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan_distance(state):
    distance = 0
    for i, val in enumerate(state):
        if val != 0:
            goal_x, goal_y = (val - 1) // 3, (val - 1) % 3
            curr_x, curr_y = i // 3, i % 3
            distance += abs(goal_x - curr_x) + abs(goal_y - curr_y)
    return distance

def get_neighbors(state):
    neighbors = []
    idx = state.index(0)
    x, y = idx // 3, idx % 3
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

Output

```
Solving 8-Puzzle...
Initial State:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Solved in 3 moves!
Step 0:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Step 1:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
```